LEARN BEING FORWARD

Programming — C# Programming

C# syntax & .NET basics

COMPLETE LEARNING GUIDE

What You'll Learn

C# runs on the .NET platform, which means understanding a few platform-level ideas — the Common Language Runtime (CLR), Intermediate Language (IL), and just-in-time (JIT) compilation — pays off just as much as learning C# syntax itself, since they explain behavior C developers coming from C/C++ often find surprising, like automatic garbage collection and runtime type checking. This sub-module covers that compilation/execution model, basic program structure, the crucial distinction between value types and reference types (which governs copying, equality, and null-ability throughout the language), the built-in data types, string handling, and C#'s modern null-safety features — all foundational to everything built in the following sub-modules on OOP, collections, and application structure.

1. The .NET Platform: CLR, IL, and JIT

C# source code compiles not directly to machine code but to Intermediate Language (IL) — a CPU-independent bytecode stored in an assembly (a .dll or .exe file) — which the Common Language Runtime (CLR) then compiles to actual machine code at runtime via a Just-In-Time (JIT) compiler, typically the first time each method is actually called. This two-stage model is what lets the same compiled assembly run on any platform with a compatible .NET runtime installed, and is also what enables the CLR's automatic memory management (garbage collection), runtime type safety checks, and exception handling — none of which C or C++ provide automatically, since those compile all the way to native machine code with no managed runtime underneath.

- An assembly (.dll/.exe) contains IL, not native machine code — the actual machine code is generated on the fly by the JIT at runtime.
- Garbage collection means you generally never call anything equivalent to C's free() or C++'s delete for ordinary managed objects — the CLR reclaims memory automatically once nothing references an object.

2. Basic Program Structure

A C# program is organized into namespaces (for organizing related types and avoiding name collisions, directly analogous to C++ namespaces) containing classes, and every executable program needs exactly one entry point — traditionally a static Main method, though modern C# (10+) also supports 'top-level statements' that let a simple program skip the explicit class/Main boilerplate entirely for small programs and scripts. Every statement ends with a semicolon, blocks are delimited with braces, and C#'s syntax for control flow (if/else, for, while, switch) closely mirrors C and C++, making the transition from those languages comparatively smooth

at the syntax level even though the runtime model underneath is very different.

```csharp
// Modern C# "top-level statements" style — no explicit class or Main needed
Console.WriteLine("Hello, world!");

// Traditional equivalent:
// using System;
// namespace MyApp {
//     class Program {
//         static void Main(string[] args) {
//             Console.WriteLine("Hello, world!");
//         }
//     }
// }
```

3. Value Types vs. Reference Types

C#'s built-in numeric types, bool, char, and struct-based types are value types, stored directly wherever they're declared (a local variable, a field) and COPIED in full whenever assigned to another variable or passed to a function — modifying the copy never affects the original. Classes, arrays, and strings are reference types, where the variable actually holds a reference (an address) to the object's data on the managed heap, so assigning one reference-type variable to another copies only the reference, meaning both variables now point to the SAME underlying object, and modifying it through either variable affects both. This distinction governs copying, equality comparisons, and whether a variable can be meaningfully null (all reference types can be null; value types cannot, unless explicitly marked nullable).

- int a = 5; int b = a; b++; leaves a unchanged — a and b are independent copies.
- A class instance assigned to a second variable shares the same object — modifying it through either variable is visible through both.

```csharp
struct PointStruct { public int X, Y; } // value type
class PointClass { public int X, Y; }   // reference type

PointStruct s1 = new PointStruct { X = 1, Y = 2 };
PointStruct s2 = s1; // full COPY
s2.X = 99;
Console.WriteLine(s1.X); // 1 — s1 is untouched

PointClass c1 = new PointClass { X = 1, Y = 2 };
PointClass c2 = c1; // same OBJECT, two references to it
c2.X = 99;
Console.WriteLine(c1.X); // 99 — c1 and c2 refer to the same object
```

4. Data Types and var vs. Explicit Typing

C#'s built-in value types include int, long, float, double, decimal (for exact base-10 arithmetic, preferred for money), bool, and char, each with a fixed, platform-independent size guaranteed by the language specification (unlike C's platform-dependent int size). The var keyword lets the compiler infer a local variable's type from its initializer expression at compile time — var count = 5; is exactly equivalent to int count = 5; with identical type safety, since var is resolved entirely at compile time, not a dynamic or loosely-typed construct the way it might appear. Style guidance generally favors var when the type is obvious from the right-hand side (reducing visual noise) and explicit typing when it improves readability for a non-obvious expression.

- decimal is preferred over double/float for monetary values specifically because it represents base-10 fractions exactly, avoiding binary floating-point rounding error.
- var is resolved to a concrete, fixed type at compile time — it is NOT the same as a dynamically typed variable.

5. String Handling and Interpolation

C# strings are immutable reference types — every operation that appears to 'modify' a string (concatenation, replacement) actually produces a brand-new string object, leaving the original unchanged, which is why repeated concatenation in a loop should use StringBuilder instead of += to avoid allocating a new string on every iteration. String interpolation ($"text {expression} more text") embeds expressions directly inside a string literal and is the modern, preferred alternative to manual string.Format or + concatenation, since the embedded expressions are visually right next to the text they produce.

```csharp
string name = "Priya";
int age = 28;

// Interpolated string — preferred modern style
string message = $"{name} is {age} years old.";
Console.WriteLine(message);

// Building a large string efficiently in a loop
var sb = new System.Text.StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.Append(i).Append(" ");
}
Console.WriteLine(sb.ToString()); // 0 1 2 3 4
```

6. Nullable Types and Null-Safety

Value types cannot normally be null, but appending ? (int? age = null;) makes a 'nullable value type' that can represent either a real value or the absence of one, commonly needed for optional database fields or form inputs that might be empty. Reference types (nullable by default historically) can opt into 'nullable reference types' (enabled per-project via <Nullable>enable</Nullable>), which has the compiler warn when a reference-type variable not marked with ? might be assigned or read as null without a check — a major modern C# feature for catching a large class of NullReferenceException bugs at compile time instead of

at runtime. The null-conditional operator (?.) and null-coalescing operator (??) provide concise, safe ways to work with potentially-null values.

```csharp
int? maybeAge = null;
Console.WriteLine(maybeAge ?? -1); // ?? provides a fallback: -1

string? name = null;
int? length = name?.Length; // ?. short-circuits to null instead of throwing
Console.WriteLine(length ?? 0); // 0
```

7. Operators and Control Flow Basics

C#'s operator set closely mirrors C/C++ (arithmetic, relational, logical, bitwise) with a few useful additions: ?? (null-coalescing, provides a default for a null value), ?. (null-conditional member access), and the pattern-matching-aware switch expression (a more concise, expression-based alternative to the traditional switch statement, introduced in C# 8). Control flow statements (if/else, for, foreach, while, do-while) work essentially as in C/C++, with foreach added specifically for iterating any collection implementing IEnumerable without manually managing an index or iterator.

```csharp
int day = 3;
string dayType = day switch { // switch EXPRESSION (C# 8+), not a switch statement
    1 or 7 => "Weekend",
    >= 2 and <= 6 => "Weekday",
    _ => "Invalid"
};
Console.WriteLine(dayType); // Weekday

foreach (int n in new[] { 1, 2, 3 }) {
    Console.Write(n + " "); // 1 2 3
}
```

Common Pitfalls

- Assuming a struct (value type) behaves like a class when assigned or passed to a method — it's fully copied, not shared by reference.
- Using += to build a large string inside a loop instead of StringBuilder, allocating a new string object on every single iteration.
- Treating var as a dynamically-typed variable instead of understanding it's resolved to one fixed, concrete type at compile time.
- Forgetting that a nullable value type (int?) must be unwrapped (via .Value, ??, or a null check) before being used where a non-nullable int is expected.

- Using double for monetary calculations instead of decimal, introducing binary floating-point rounding error into currency amounts.
- Dereferencing a reference-type variable without a null check (or ?.) when nullable reference types are enabled and the compiler has already warned about it.

Key Takeaways

- C# compiles to IL, which the CLR JIT-compiles to native code at runtime — this managed runtime is what enables automatic garbage collection.
- Value types are copied on assignment/passing; reference types share the same underlying object through every reference to it.
- var is resolved to a fixed, concrete type at compile time — it is not loosely typed or dynamic.
- Use decimal (not float/double) for any monetary or exact base-10 calculation.
- Nullable value types (T?) and nullable reference types (enabled via project settings) let the compiler catch a large class of null-related bugs before runtime.

Learn More

Microsoft Learn — C# language reference
https://learn.microsoft.com/en-us/dotnet/csharp/

Microsoft Learn — Value types and reference types
https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/value-reference-types